



초기 설정

MAC OS 2019 PRO

- 안드로이드 스튜디오 intel 86_48 설치
 - PIXEL 4 XL 설치
 - 아래 경로에서 `.apk` 다운로드

`dreamhack-lecture.s3.ap-northeast-2.amazonaws.com`

`https://dreamhack-lecture.s3.ap-northeast-2.amazonaws.com/uploads/mobile-android/DreamDetector.apk`

- PIXEL 4 XL 실행
 - 터미널 이동
 - `adb devices`
 - 에뮬레이터 실행 확인 명령어
 - `adb install -r DreamDetector.apk`
 - `.apk` 파일 설치
 - `cd ~/Downloads`
 - 설치 경로 이동
 - `adb shell pm list packages | grep -i dream`
 - 설치 파일명 확인 명령어
 - `package:android.com.dream_detector` 라는 설치 파일명 확인

```
[mac@macui-MacBookPro ~ % adb shell pm list packages | grep -i dream
package:com.android.dreams.basic
package:android.com.dream_detector
```

- `adb shell pm path android.com.dream_detector`

- 해당 설치 파일 경로 확인 명령어

- `package:/data/app/~~isePQg0e7B_APJcOieNKCQ==/android.com.dream_detector-4AE6cG-1cNEz0d8A-6k_tw==/base.apk` 라는 파일 설치 경로 확인

```
mac@macui-MacBookPro ~ % adb shell pm path android.com.dream_detector
package:/data/app/~~isePQg0e7B_APJcOieNKCQ==/android.com.dream_detector-4AE6cG-1cNEz0d8A-6k_tw==/base.apk
mac@macui-MacBookPro ~ % adb shell ls /data/app/~~isePQg0e7B_APJcOieNKCQ==/android.com.dream_detector-4AE6cG-1cNEz0d8A-6k_tw==/base.apk
/data/app/~~isePQg0e7B_APJcOieNKCQ==/android.com.dream_detector-4AE6cG-1cNEz0d8A-6k_tw==/base.apk
```

- `adb shell ls /data/app/~~isePQg0e7B_APJcOieNKCQ==/android.com.dream_detector-4AE6cG-1cNEz0d8A-6k_tw==/base.apk`

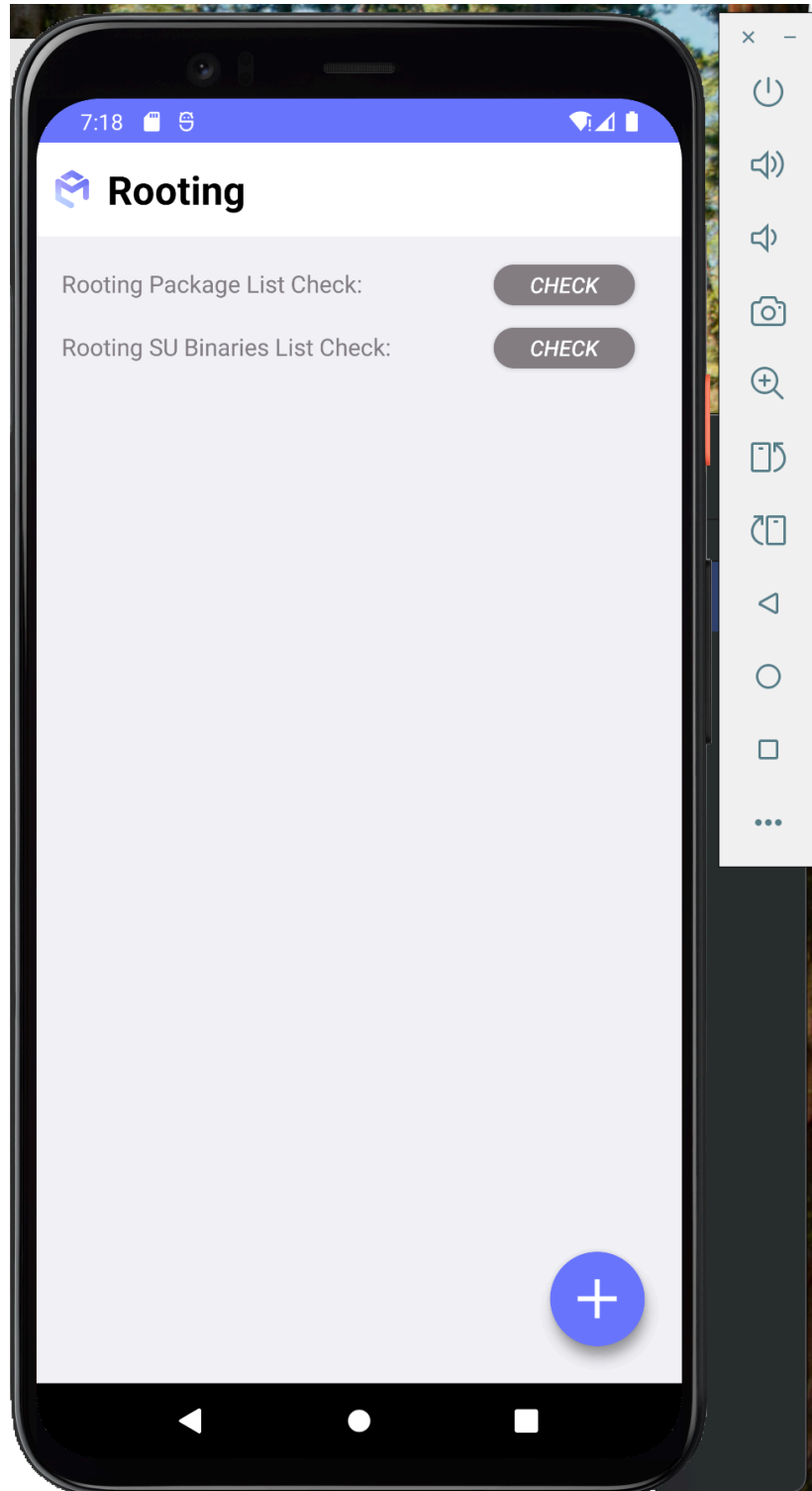
- 안드로이드에 해당 경로의 파일 쉘 가져옴

- `adb shell monkey -p android.com.dream_detector 1`

- 해당 파일 쉘 실행

```
mac@macui-MacBookPro ~ % adb shell monkey -p android.com.dream_detector 1
bash arg: -p
[ bash arg: android.com.dream_detector
  bash arg: 1
args: [-p, android.com.dream_detector, 1]
arg: "-p"
arg: "android.com.dream_detector"
arg: "1"
data="android.com.dream_detector"
Events injected: 1
## Network stats: elapsed time=58ms (0ms mobile, 0ms wifi, 58ms not connected)
mac@macui-MacBookPro ~ %
```

- 아래와 같이 `rooting` 실행되며 우하단 `+` 클릭



Android Emulator에 접속 (Server-side Frida 설치)

- ```
mac@macui-MacBookPro Downloads % python3 - << 'EOF'
heredoc> import lzma, shutil
with lzma.open("frida-server-17.5.2-android-x86_64.xz") as f_in:
 with open("frida-server-17.5.2-android-x86_64", "wb") as f_out:
 shutil.copyfileobj(f_in, f_out)
EOF
```

- ```
mac@macui-MacBookPro Downloads % adb push frida-server-17.5.2-android-x86_64 /data/local/tmp/frida-server
adb shell chmod 755 /data/local/tmp/frida-server
adb shell /data/local/tmp/frida-server &
```

```
mac@macui-MacBookPro Downloads % adb shell ps -A | grep frida
```

```
mac@macui-MacBookPro Downloads % frida -U -f android.com.dream_detector
```

```
mac@macui-MacBookPro Downloads % frida -U -f android.com.dream_detector
```

```

    /----|
    | _- |   Frida 17.5.2 - A world-class dynamic instrumentation toolkit
    | (-| |
    > _- |   Commands:
    /_/_|-|     help      -> Displays the help system
    . . . .     object?   -> Display information about 'object'
    . . . .     exit/quit -> Exit
    . . . .
    . . . . More info at https://frida.re/docs/home/
    . . . .
    . . . . Connected to Android Emulator 5554 (id=emulator-5554)
Spawned `android.com.dream_detector`. Resuming main thread!
[Android Emulator 5554::android.com.dream_detector ]-> █
```



FRIDA 기본 명령어

- DBI (Dynamic Binary Instrumentation)
 - 바이너리 프로그램이 실행되는 동안 명령어를 수정, 삽입 하는 것
 - 목적
 - 프로세스 분석
 - 추적
 - 디버깅

기능	설명
Injection	임의 코드를 메모리나 애플리케이션에 주입합니다.
Interception	특정 함수를 후킹하여 매개 변수나 반환 값을 수정하고, 함수를 재작성합니다.
Stalking	애플리케이션에서 명령어 레벨의 코드를 추적하여 디버깅하거나 메모리 내의 데이터를 검색, 추출합니다.

FRIDA 기본 명령어

명령어	설명
frida	앱을 실행하여 frida와 애플리케이션을 연결, 스크립트 로드
frida-ps	프로세스 목록 출력
frida-trace	함수 호출 추적
frida-ls-devices	연결된 기기 목록 출력
frida-kill	동작중인 프로세스 종료

기능	옵션	설명
사용법	<code>-h, --help</code>	옵션별 상세 사용법 출력
기기 연결	<code>U</code> , <code>-usb</code>	USB로 연결된 기기를 연결
	<code>-D <DEVICE ID></code>	가상 기기를 연결

기능	옵션	설명
Attach	<code>-F</code>	현재 foreground에서 실행되고 있는 앱을 연결
	<code>-n <PACKAGE NAME></code>	앱의 패키지 이름으로 앱을 연결
	<code>-p <PID></code>	앱의 프로세스 ID로 앱을 연결
Spawn	<code>-f <PACKAGE NAME></code>	앱을 실행하고, 연결

USB로 연결된 기기에 frida Attach

```
$ frida -U -F
```

```

____
/_ _ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit
|(_)|
> _ | Commands:
/_/_|_ | help    → Displays the help system
....  object? → Display information about 'object'
....  exit/quit → Exit
....
....  More info at https://frida.re/docs/home/
[iPad::Cydia] →
```

가상기지에서 캘린더 앱 Spawn

```
$ frida -D emulator-5554 -f com.google.android.calendar
```

```

  ____
 / _ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit
| ( _ |
> _ |  Commands:
/_/ |_ |   help    → Displays the help system
....   object?   → Display information about 'object'
....   exit/quit → Exit
....
....   More info at https://frida.re/docs/home/
Spawned `com.google.android.calendar`. Resuming main thread!
[Android Emulator 5554::com.google.android.calendar] →
```

Frida 스크립트 파일 로드 방법

기능	옵션	설명
Load Script	<code>-l <SCRIPT></code>	frida script 파일을 로드

```
$ echo "console.log(\"script loaded.\");" > script.js
$ frida -U -F -l script.js
```

```

  ____
 / _ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit
| ( _ |
> _ |  Commands:
/_/ |_ |   help    → Displays the help system
....   object?   → Display information about 'object'
....   exit/quit → Exit
....
```

```
.... More info at https://frida.re/docs/home/  
script loaded.  
[iPad::Cydia]→
```

frida-trace

- 함수 호출을 동적 추적
- 공용 옵션 사용해 연결 기기 지정
- 분석 대상 앱에 연결 기기 부착

추적 필터링 명령어

- **include**
 - 추적 원하는 모듈 추가
- **exclude**
 - 추적 원치 않는 모듈 제거

기능	옵션	설명
Attach	<code>-F</code>	현재 foreground에서 실행되고 있는 앱을 연결
	<code>-n <PACKAGE NAME></code>	앱의 패키지 이름으로 앱을 연결
	<code>-p <PID></code>	앱의 프로세스 ID로 앱을 연결
Spawn	<code>-f <PACKAGE NAME></code>	앱을 실행하고, 연결
Include/Exclude	<code>-I <MODULE></code>	모듈의 모든 함수를 추적
	<code>-X <MODULE></code>	모듈의 모든 함수를 추적에서 제외
	<code>-i <MODULE!FUNCTION></code>	함수를 추적(glob pattern)
	<code>-x <MODULE!FUNCTION></code>	함수를 추적에서 제외(glob pattern)
	<code>-a <FUNCTION!offset></code>	함수를 추적(offset)

가상 기기의 chrome앱에서 recv* 와 send* 함수 추적

```
$ frida-trace -D emulator-5554 -f com.android.chrome -i "recv*" -i "send*"
```

Instrumenting...

recv: Loaded handler at "/home/user/libc.so/recv.js"

recvfrom: Loaded handler at "/home/user/libc.so/recvfrom.js"

recvmsg: Loaded handler at "/home/user/libc.so/recvmsg.js"

recvmsg: Loaded handler at "/home/user/libc.so/recvmsg.js"

sendfile64: Loaded handler at "/home/user/libc.so/sendfile64.js"

sendmsg: Loaded handler at "/home/user/libc.so/sendmsg.js"

send: Loaded handler at "/home/user/libc.so/send.js"

sendfile: Loaded handler at "/home/user/libc.so/sendfile.js"

sendmsg: Loaded handler at "/home/user/libc.so/sendmsg.js"

sendto: Loaded handler at "/home/user/libc.so/sendto.js"

Started tracing 10 functions. Press Ctrl+C to stop.

```
/* TID 0x7f05 */
```

```
1959 ms recvfrom() [libc.so]
```

```
/* TID 0x7f38 */
```

```
2059 ms recvfrom() [libc.so]
```

```
2062 ms recvfrom() [libc.so]
```

```
/* TID 0x7f05 */
```

```
2085 ms recvfrom() [libc.so]
```

```
2085 ms recvfrom() [libc.so]
```

```
2089 ms recvfrom() [libc.so]
```

```
3569 ms recvfrom() [libc.so]
```

```
3570 ms recvfrom() [libc.so]
```

```
3576 ms recvfrom() [libc.so]
```

```
/* TID 0x7f53 */
```

```
4196 ms sendmsg() [libc.so]
```

```
/* TID 0x7f05 */
```

```
4211 ms recvfrom() [libc.so]
```

```
4211 ms recvfrom() [libc.so]
```

```
4211 ms recvfrom() [libc.so]
```

```
4212 ms sendto() [libc.so]
```

```
/* TID 0x7f53 */
```

```
4281 ms recvmsg() [libc.so]
```

```
...
```

연결 가능 기기 목록 출력 명령어

```
frida-ls-devices
```

```
$ frida-ls-devices
Id      Type  Name
-----
local   local  Local System
emulator-5554 usb    Android Emulator 5554
socket  remote Local Socket
```

프로세스 종료 명령어

```
frida-kill -D <DEVICE-ID> <PID>
```

```
$ frida-ps -D emulator-5554 -a
PID Name      Identifier
-----
27157 Chrome    com.android.chrome
21430 Drive     com.google.android.apps.docs
1836  Google     com.google.android.googlequicksearchbox
1836  Google     com.google.android.googlequicksearchbox
21468 Messages  com.google.android.apps.messaging
26485 YouTube   com.google.android.youtube
$ frida-kill -D emulator-5554 27157
$ frida-ps -D emulator-5554 -a
PID Name      Identifier
-----
21430 Drive     com.google.android.apps.docs
8047  Google     com.google.android.googlequicksearchbox
8047  Google     com.google.android.googlequicksearchbox
```

21468 Messages com.google.android.apps.messaging

26485 YouTube com.google.android.youtube



FRIDA 객체

Process

- 프로세스 정보 조회 객체
- 프로세스가 속한 메모리 조회
 - PID
 - 페이지

모든 프로세스 출력

```
Process.enumerateModules()
```

첫번째 인자에 전달된 주소가 어떤 모듈에 할당된 메모리인지 확인

```
Process.findModuleByAddress(address)  
Process.getModuleByAddress(address)
```

첫번째 인자에 전달된 모듈 이름의 주소 출력

```
Process.findModuleByName(name)  
Process.getModuleByName(name)
```

전달된 메모리의 페이지 크기와 권한 출력

```
Process.findRangeByAddress(address)
Process.getRangeByAddress(address)
```

Process 함수 사용 예시

```
var modules = Process.enumerateModules()[0];
```

```
console.log("modules Base: ", modules.base);
console.log("modules Path: ", modules.path);
```

```
var modules_info = Process.findRangeByAddress(ptr(modules.base))
```

```
console.log("Protection: ", modules_info.protection);
```

Thread

- 현재 스레드에서 백트레이스 출력
- 실행 일시 중지 기능
- 백트레이스
 - 앱 분석 시 발생하는 버그 확인
 - 앱 분석시 실행 흐름 확인

현재 스레드의 백트레이스 제공

```
Thread.backtrace([context, backtracer])
```

Thread 백트레이스 제공 함수 예시

```
console.log('called `from:\n' +
  Thread.backtrace(this.context, Backtracer.FUZZY).map(DebugSymbol.fromAddress).join('\n') + '\n');
```

현재 쓰레드의 실행 중지 목적 사용되는 시계 (초 단위)

```
Thread.sleep(delay)
```

캘린더 앱에 Thread 함수 스크립트 실행

```
$ frida -D emulator-5554 -f com.google.android.calendar -l thread.js
```

```

____
/_ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit
|(_|
>_ | Commands:
/_|_| help    → Displays the help system
.... object? → Display information about 'object'
.... exit/quit → Exit
....
.... More info at https://frida.re/docs/home/
Spawning `com.google.android.calendar`...
called from:
0x74f882a25a23 frida-agent-64.so!0x29fa23
0x74f882a6fc9b frida-agent-64.so!0x2e9c9b
0x74f882a7b758 frida-agent-64.so!0x2f5758
0x74f882a7cf75 frida-agent-64.so!0x2f6f75
0x74f882a85feb frida-agent-64.so!0x2fffeb
0x74f882a85f65 frida-agent-64.so!0x2fff65
0x74f882a16618 frida-agent-64.so!0x290618
0x74f882917b97 frida-agent-64.so!0x191b97
0x74f88297a562 frida-agent-64.so!0x1f4562
0x74f88297a7f8 frida-agent-64.so!0x1f47f8
0x74f88297a967 frida-agent-64.so!0x1f4967
0x74f882a14828 frida-agent-64.so!0x28e828
0x74f882985d7c frida-agent-64.so!0x1ffd7c
```

0x74f91ce94bac libc.so!_ZL15__pthread_startPv+0x1c

0x74f91ce2cf2e libc.so!__start_thread+0x3e

Spawned `com.google.android.calendar`. Resuming main thread!

[Android Emulator 5554::com.google.android.calendar]→

Memory

- 메모리에 동작하는 행위 정보 제공 객체
- 정보 제공 리스트
 - 메모리 할당
 - `Memory.alloc(size[, options])`
 - 프로세스 힙 메모리 내 전달된 크기만큼 메모리 할당
 - 주소를 `NativePointer` 형태로 반환
 - `Memory.allocUtf8String(str)`
 - 프로세스 힙 메모리 내 UTF8 형태 문자열 할당
 - 주소를 `NativePointer` 형태로 반환
 - 메모리 검색
 - `Memory.scan(address, size, pattern, callbacks)`
 - 특정 주소에서 원하는만큼 메모리 스캔 및 바이트 출력
 - callback 함수 등록하여 특정 바이트 매칭 시 바이트 내역과 스캔 종료 시 코드를 구성
 - `Memory.scanSync(address, size, protection)`
 - 특정 주소에서 원하는 만큼 메모리 스캔 및 바이트 출력 (동기 방식)
 - 메모리 복사
 - 메모리 패치
 - 메모리 권한 변경
 - `Memory.protect(address, size, protection)`

Memory 함수 예시

```
var m = Process.enumerateModules()[0];
//현재 프로세스 모든 모듈 출력

var pattern = "2f 6c 69 62" // "/lib"
var perm = 0;

Memory.scan(m.base, m.size, pattern, {
  // m.base부터 m.size만큼의 범위를 pattern = /lib 으로 바이트 단위 검색
  onMatch: function(address, size) {
    // 패턴이 발견된 메모리 주소와 매칭된 바이트 크기를 찾았을때 실행되는 콜백
    console.log('Found match at ' + address);
    // /lib이 발견된 실제 메모리 주소 출력

    // get protection
    perm = Process.findRangeByAddress(address);
    // 해당 주소가 포함된 메모리 페이지 범위를 perm 변수에 복사
    console.log("Protection: " + perm.protection);
    // 현재 메모리 권한 출력

    // change protection
    Memory.protect(address, 4096, 'rw-');
    // 해당 주소가 속한 4096bytes를 읽기 + 쓰기 가능으로 변경
  },
  onComplete: function () {
    console.log('Done');
    // 메모리 스캔 종료 시 'Done' 출력
  }
})

var string = Memory.allocUtf8String("Dreamhack");
// 프로세스 힙 영역에 존재하는 "Dreamhack" 문자열을 UTF-8로 String 변수에 할당
console.log('Allocated string address: ' + string);
// 해당 문자열이 있는 메모리 주소를 출력
```



```
var pattern2 = "68 61 63 6b"; // pattern2 변수에 "hack" 입력
var result = Memory.scanSync(string, 0x100, pattern2);
// String 변수부터 0x100 바이트 범위 안에서 "hack" 패턴을 동기 방식으로 스캔
console.log("Memory.scanSync() result: " + JSON.stringify(result));
// 스캔 결과를 JSON 형식으로 출력
```

앱 메인 모듈에서 /lib 문자열을 찾아 해당 메모리 페이지에 읽기+쓰기 권한 부여 후,
새 문자열을 힙에 할당한 뒤 그 안에서 "hack"을 다시 메모리 스캔 하는 코드

실행 결과

```
$ frida -D emulator-5554 -f com.google.android.calendar -l memory.js

___
/_ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit
|(_|
>_ | Commands:
/_|_| help    → Displays the help system
.... object? → Display information about 'object'
.... exit/quit → Exit
....
.... More info at https://frida.re/docs/home/
Spawning `com.google.android.calendar`...
Allocated string address: 0x7b539b838670
Memory.scanSync() result: [{"address":"0x7b539b838675","size":4}]
Found match at 0x5b13a8a2fd5c
Protection: r--
Done
Spawnd `com.google.android.calendar`. Resuming main thread!
[Android Emulator 5554::com.google.android.calendar]→
```

Module

- 프로세스 주소 파악 용도 사용 객체
 - 프로세스의 베이스 주소 복사

```
Module.findBaseAddress(name)
Module.getBaseAddress(name)
```

- 외부 함수 주소 복사

```
Module.findExportByName(moduleName|null,exportName)
Module.getExportByName(moduleName|null,exportName)
```

- Module 함수 예시

```
var fopen = Module.findExportByName(null, "fopen");
// 라이브러리(모듈)의 export 함수의 주소 찾아서 fopen 변수에 복사
// 첫번째 인자가 null이므로 어떤 모듈이든 상관 없이 현재 프로세스에
// 로드된 것 중 fopen이라는 export를 복사
console.log("fopen: " + fopen);
// 위에서 찾은 fopen의 주소를 콘솔에 출력

// iOS
var mName = "libsystem_c.dylib"

// Android
var mName = "libc.so"
// 파일명이 libc.so인 파일을 mName에 복사

var fopen_m = Module.findExportByName(mName, "fopen");
// null이 아닌 mName에서만 fopen을 찾음
console.log("fopen with module: " + fopen_m);
// 특정 모듈에서 찾은 fopen 주소를 출력
```

- 실행 결과

```
$ frida -D emulator-5554 -f com.google.android.calendar -l module.js
```

```
____
/_ _ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit
|(_)|
> _ | Commands:
```

```
/_/_|_|    help    → Displays the help system
....      object?  → Display information about 'object'
....      exit/quit → Exit
....
....      More info at https://frida.re/docs/home/
Spawning `com.google.android.calendar`...
fopen: 0x74f91ce8d5e0
fopen with module: 0x74f91ce8d5e0
Spawned `com.google.android.calendar`. Resuming main thread!
[Android Emulator 5554::com.google.android.calendar]→
```



FRIDA 데이터 타입

Int64 / Uint64

`new Int64(v)` , `int64` , `uint64`

인자에 문자열 또는 16진수 형태의 값이 전달되면 10진수로 변환하여 연산 수행

- `int64`
 - 음수 표현 불가
- `uint64`
 - 음수 표현

연산 함수

- **`add(rhs)`**
 - 덧셈
- **`sub(res)`**
 - 뺄셈
- **`and(res)`**
 - and 연산
- **`or(res)`**
 - or 연산
- **`xor(res)`**
 - xor 연산

시프트 함수

- 변환된 10진수 값에 대해 n 비트 만큼 쉬프트 연산 수행

- **shr(n)**
 - 오른쪽으로 n칸 시프트
- **shl(n)**
 - 왼쪽으로 n칸 시프트

비교 함수

- **compare(rhs)**
 - 인자로 전달된 값 비교

타입 변경 → 숫자

- **toNumber()**
 - Int64 또는 Uint64를 사용한 정수 객체를 숫자 타입 변경

타입 변경 → 문자열

- **toString([radix=10])**
 - 변환된 10진수. 값을 문자열 형태로 변환
 - radix
 - 의미 : 진수
 - 기본값. 10

```
function add(num, num2) {
  console.log("type: " + typeof(num) + ", " + typeof(num2));
  // num, num2를 더하여 console.log로 출력
  // typeof(num)은 num의 타입을 출력 반환
  // 문자열 반환 이유 : 값 출력
  var num_obj = int64(num);
  var num_obj2 = int64(num2);
  // int64(num) 통해 64비트 정수로 변환
  // 큰 정수 연산 시 사용
  var sum = num_obj.add(num_obj2);
}
```

```

//
console.log("add sum: " + sum + " type: " + typeof(sum));
}
// 덧셈 결과와 그 타입을 출력
function compare(num, num2) {
// compare 함수 또한 내부 숫자 2개(num, num2)를 64비트 정수로 변환하여 비교
준비
    var num_obj = int64(num);
    var num_obj2 = int64(num2);
// 64비트 정수로 변환하여 비교 준비
    var result = num_obj.compare(num_obj2);
// 두개의 64비트 정수를 비교하여 참 거짓에 따라 result 인자에 -1, 0, 1 반환
    if(!result) {
        console.log("Same.")
    }
// 비교 결과가 0이면, same 출력
    else {
        console.log("Different.");
    }
}
// 비교결과가 1 또는 -1이면 Different 출력

function convert(num) {
    var num_obj = int64(num);
// convert 함수 또한 숫자를 64비트 정수로 변환하는 함수
// int64(num)로 num을 64비트 정수로 변환
    console.log("Decimal: " + num_obj.toString(10));
    console.log("Hexadecimal: " + num_obj.toString(16));
}
// 64비트 정수를 10진수 문자열로 변환
// 64비트 정수를 16진수 문자열로 변환

add("100", "200");
compare("100", "200");
compare("100", "100");
convert("1234");

```

```
$ frida -D emulator-5554 -f com.google.android.calendar -l int64.js
```

```
_____  
/_|_ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit  
|(_| |
```

```
>_ |_ | Commands:
```

```
/_/_|_ | help → Displays the help system
```

```
.... object? → Display information about 'object'
```

```
.... exit/quit → Exit
```

```
....
```

```
.... More info at https://frida.re/docs/home/
```

```
Spawning `com.google.android.calendar`...
```

```
type: string, string
```

```
add sum: 300 type: object
```

```
Different.
```

```
Same.
```

```
Decimal: 1234
```

```
Hexadecimal: 4d2
```

```
Spawned `com.google.android.calendar`. Resuming main thread!
```

```
[Android Emulator 5554::com.google.android.calendar] →
```

NativePointer

인자로 전달된 문자열 값 → 메모리 포인터로 변환

`new NativePointer(s)` 또는 `ptr` 형태로 사용

인자 `s` 는 10진수 혹은 16진수 형태 메모리 주소여야 함

isNull()

- 포인터 NULL 여부 검사

toInt32()

- NativePointer 타입을 signed 32-bit 정수로 캐스팅

readPoint() & writePointer(ptr)

- 포인터 주소에서 NativePointer 형태로 값을 읽기/쓰기

readS8() & readU8() & readFloat() & readDouble()

- 포인터 주소로부터 각 함수의 접미사를 타입으로 하여 값 읽기
- ex) readU16 함수 포인터 주소로부터 부호 없는 16비트 값 읽기

writeS8() & writeU8() & writeShort() & writeLong()

- 포인터 주소에 각 함수의 접미사를 타입으로 하여 값 쓰기
- ex) writeU16 함수는 포인터 주소에 부호 없는 16비트 값 쓰기

readByteArray(length)

- 포인터 주소로부터 length 길이 만큼 값 읽기

writeByteArray(bytes)

- ArrayBuffer 타입의 배열을 포인터 주소 쓰기

readCString([size = -1]), readUtf8String([size = -1])

- 인자로 전달한 size만큼 문자열 읽기
- 각 함수에 따라 ASCII, UTF-8 문자열 형태로 읽기
- size값이 -1일 경우, NULL 값이 나타날때까지 읽기

writeUtf8String(str), writeAnsiString(str)

- 인자로 전달한 문자열을 UTF-8, ANSI 문자열 형태로 포인터 주소 쓰기
- writeAnsiString 함수는 Windows에서만 유효

예시 I


```

var str = Memory.allocUtf8String("Dreamhack");
console.log("str address: " + str);
// Memory.allocUtf8String()은 UTF-8로 인코딩된 문자열 "Dreamhack"을 프로
세스 힙 메모리에 할당
// str 변수는 메모리 주소를 가르키는 NativePointer 객체가 됨

console.log("str.isNull: " + str.isNull());
// str.isNull() str이 null 포인터인지 확인
// null 포인터라면 true, 아니면 false 반환
console.log("str.add(4): " + str.add(4));
// str.add(4) : str의 메모리 주소에 4바이트를 더한 주소를 반환
// 문자열 "Dreamhack"에서 4바이트 떨어진 위치 가르킴, 즉 h를 가르킴.
console.log("str.toString(): " + str.toString());
// str "Dreamhack"을 가르키므로, 문자열로 "Dreamhack" 출력
console.log("str.readPointer(): " + str.readPointer());
// str이 가리키는 메모리 주소에서 8바이트 포인터를 읽어 반환
console.log("str.readU16(): " + str.readU16());
// str이 가리키는 메모리 주소에서 2바이트를 읽어 부호 없는 16비트 정수로 반환
// "Dreamhack"에서 첫 2바이트(0x44 0x72) -> "Dr"가 읽혀서 출력
console.log("str.writeU16(0x4141): " + str.writeU16(0x4141));
// "0x4141이 가리키는 메모리 주소에 0x4141 값을 2바이트로 쓰기
// "AA"로 표현되어 "Dreamhack"의 첫 2개 바이트가 "AA"로 변경
console.log("str.readU16(): " + str.readU16());
// "AA"를 출력
console.log("str.readUtf8String(): " + str.readUtf8String());
// 문자열 "AAeamhack" 출력

```

결과

```

$ frida -D emulator-5554 -f com.google.android.calendar -l nativePointer1.js

```

```

┌───┐
│_ _| Frida 16.1.3 - A world-class dynamic instrumentation toolkit
│(_ _|
│> _| Commands:
│/_ _| help → Displays the help system
│... object? → Display information about 'object'
│... exit/quit → Exit

```

```

.....
..... More info at https://frida.re/docs/home/
Spawning `com.google.android.calendar`...
str address: 0x74f89a9b2000
str.isNull: false
str.add(4): 0x74f89a9b2004
str.toString(): 0x74f89a9b2000
str.readPointer(): 0x6361686d61657244
str.readU16(): 29252
str.writeU16(0x4141): 0x74f89a9b2000
str.readU16(): 16705
str.readUtf8String(): AAeamhack
Spawned `com.google.android.calendar`. Resuming main thread!
[Android Emulator 5554::com.google.android.calendar] →

```

예시 II

```

function getFuncModule(funcName) {
  // 주어진 함수 이름인 funcName 에 해당하는 모듈 경로 찾기
  var addr = Module.findExportByName(null, funcName);
  // 현재 프로세스에서 funcName 함수가 속한 모듈의 주소 반환
  // null을 전달하면, 모든 로드된 모듈에서 해당 함수 이름 찾기
  var obj = Process.findModuleByAddress(ptr(addr));
  //
  return obj.path;
}
// obj.path를 통해 모듈의 경로 반환

var libPath;
libPath = getFuncModule("open");
// open 함수가 속한 모듈 경로를 찾아 libPath에 저장
console.log("Path: " + libPath);
// libPath를 출력하여 "open"함수가 속한 모듈의 경로를 콘솔 출력
libPath = getFuncModule("fopen");
// fopen함수에 대해서도 같은 작업 반복
console.log("Path: " + libPath);
// fopen 함수가 속한 모듈의 경로를 콘솔 출력

```

결과

```
$ frida -D emulator-5554 -f com.google.android.calendar -l nativePointer2.js
```

```
____
/_ _|  Frida 16.1.3 - A world-class dynamic instrumentation toolkit
|(_ _|
> _ _|  Commands:
/_/_ _|  help    → Displays the help system
....    object? → Display information about 'object'
....    exit/quit → Exit
....
....    More info at https://frida.re/docs/home/
Spawning `com.google.android.calendar`...
Path: /system/lib64/libc.so
Path: /system/lib64/libc.so
Spawned `com.google.android.calendar`. Resuming main thread!
[Android Emulator 5554::com.google.android.calendar] →
```

NativeFunction

- 코드에서 지정한 주소를 실제 함수처럼 사용하는 객체
- 인자로 함수의 주소 & 반환 타입 & 인자 타입을 지정 시 해당 함수 호출 가능
- Frida 코드 내 라이브러리 함수 직접 호출 시 사용

```
new NativeFunction(address, returnType, argTypes[, abi])
```

타입 종류

- void
- pointer
- int, uint, long, ulong
- char, uchar

- float, double
- int8, uint8, int16, uint16, int32, uint32, int64, uint64
- bool

```

var open = new NativeFunction(Module.getExportByName(null, "open"), "int", ["pointer", "int"]);
// getExportByName : 현재 프로세스의 "open" 함수의 주소를 찾는 함수
// "int" : open 함수의 반환 타입을 int로 설정
// ["pointer", "int"] : open 함수의 파라미터로 첫번째는 파일 경로 (pointer), 두번째로 파일 모드(int) 설정
var write = new NativeFunction(Module.getExportByName(null, "write"), "ssize_t", ["int", "pointer", "size_t"]);
// write 변수는 ssize_t(파일에 쓴 바이트 수)를 반환
// int = 파일 디스크립터, pointer = 쓰기 내용, size_t = 쓰기 데이터의 크기
var close = new NativeFunction(Module.getExportByName(null, "close"), "int", ["int"]);
// close는 int를 반환하는 함수
// 파라미터는 파일 디스크립터 (int)로 해당 파일을 닫는 함수

// Android
var filePath = "/data/data/com.google.android.calendar/test.txt";
// iOS
// var filePath = "/tmp/test.txt";

var O_WRONLY = 1;
// 파일을 쓰기 전용으로 열기 위한 플래그 값
// O_WRONLY는 파일을 쓰기 모드로 열기
// 1은 쓰기 전용 모드 의미

// Android
var O_CREAT = 0x40;
// iOS
// var O_CREAT = 0x100;

var fd = open(Memory.allocUtf8String(filePath), O_CREAT|O_WRONLY);
// filePath에 해당하는 파일을 쓰기 모드(O_WRONLY)와 생성 모드 (O_CREAT)로 열

```

```

기
console.log("file descriptor: " + fd);
// 파일 디스크립터 fd를 출력.
// ex. fd 값 3
write(fd, Memory.allocUtf8String("Hello World\n"), 12);
// fd로 열린 파일에 "Hello World\n"이란 문자열 쓴다.
// 12는 쓰기 데이터 크기로, "Hello World\n"의 길이인 12바이트를 쓴다.
// 문자열 "Hello World\n"를 메모리에 할당하고 그 주소를 write()에 전달
close(fd);
// fd 파일 디스크립터로 열린 파일을 닫음
console.log("done, check " + filePath);
// "done" 메시지 출력하고 파일 경로를 콘솔에 표시
// 파일을 성공적으로 열고 쓰고 닫았다는 의미

```

결과

```
$ frida -D emulator-5554 -f com.google.android.calendar -l nativeFunction.js
```

```

┌───┐
/_ _ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit
|(_)|
> _ | Commands:
/_/_|_ | help → Displays the help system
.... object? → Display information about 'object'
.... exit/quit → Exit
....
.... More info at https://frida.re/docs/home/
Spawning `com.google.android.calendar`...
file descriptor: 52
done, check /data/data/com.google.android.calendar/test.txt
Spawned `com.google.android.calendar`. Resuming main thread!
[Android Emulator 5554::com.google.android.calendar] → exit

```

```

Thank you for using Frida!
$ adb shell
generic_x86_64_arm64:/ $ su

```

```
generic_x86_64_arm64:/ # cat /data/data/com.google.android.calendar/test.txt  
Hello World  
generic_x86_64_arm64:/ #
```



Interceptor

- 동적으로 바이너리 분석
 - 특정 함수를 명시
 - 해당 함수가 호출되면 임의 코드 실행
 - 정보 조회
 - attach 콜백 등록 가능
 - 명시한 함수가 호출되면 함수가 실행/종료 시 레지스터와 스택 등 확인 가능
 - 임의 코드 추가 후 실행 가능
- **Interceptor.attach(target, callbacks[, data])**
 - target에 해당 함수 호출 시 등록된 콜백에 따라 임의 코드 실행
 - 함수 인자 조회 등 실행 흐름 파악
 - 함수 인자 조작 등 실행 흐름 파악
- **Interceptor.replace(target, replacement[, data])**
 - target 해당 함수 호출 시 함수 기능 미실행 및 임의 코드 실행

open 함수 주소 파악 & attach 함수의 인자 전달 & open 함수 실행 시 onEnter 콜백 함수에서 2개 인자 출력

- 출력 방법
 - args 배열과 this.context 내 레지스터 이름 이용

```
var open = Module.findExportByName(null, "open");  
// 현재 프로세스에서 open이라는 이름 가진 함수 찾기  
Interceptor.attach(open, {
```

```

// open 함수 호출 시 해당 함수 실행 가로챈
// 첫번째 인자는 가로채고싶은 함수 명. 두번째 인자는 가로챈 후 실행할 콜백명
onEnter(args) {
    console.log("Filename: " + Memory.readUtf8String(args[0]));
    // onEnter는 함수 호출 시 실행되는 콜백, arg는 함수 인자
    // 해당 메모리 위치의 UTF-8 문자열 읽어 반환
    // Android
    // console.log("Filename: " + args[0].readCString() + ", " + this.context.x0.readCString()); // iOS
    console.log("flags: " + args[1].toInt32()); // Android
    // console.log("flags: " + args[1].toInt32() + ", " + this.context.x1); // iOS
    OS
},
// open 함수의 두번째 인자(arg[1])는 플래그 값
// toInt32()는 이 값을 32비트 정수로 변환 및 출력
// 즉 Android에서 플래그 값 출력하는 코드
onLeave(retval) {
    console.log("ret: " + retval);
}
});
// 함수 종료시키는 콜백
// retval은 open 함수의 반환값이며 open 함수 성공 여부를 콘솔에 출력

```

결과

```
$ frida -D emulator-5554 -f com.google.android.calendar -l interceptor.js
```

```

┌───┐
/_ _ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit
|(_)|
> _ | Commands:
/_/ |_ | help → Displays the help system
.... object? → Display information about 'object'
.... exit/quit → Exit
....
.... More info at https://frida.re/docs/home/
Spawned `com.google.android.calendar`. Resuming main thread!

```



```

[Android Emulator 5554::com.google.android.calendar] → Filename: /proc/s
elf/cmdline, /proc/self/cmdline
flags: 524288, 0x80000
ret: 0x28
Filename: /system/framework/org.apache.http.legacy.boot.jar.x86_64.flock,
/system/framework/org.apache.http.legacy.boot.jar.x86_64.flock
flags: 66, 0x42
ret: 0xffffffffffffff
Filename: /system/framework/oat/x86_64/org.apache.http.legacy.boot.vde
x, /system/framework/oat/x86_64/org.apache.http.legacy.boot.vdex
flags: 0, 0x0
ret: 0x28
Filename: /data/dalvik-cache/x86_64/system<at:var at:name="framework"
/>boot.art, /data/dalvik-cache/x86_64/system<at:var at:name="framewor
k" />boot.art
flags: 0, 0x0
ret: 0x28
Filename: /data/dalvik-cache/x86_64/system<at:var at:name="framework"
/>boot-core-libart.art, /data/dalvik-cache/x86_64/system<at:var at:name
="framework" />boot-core-libart.art
flags: 0, 0x0
ret: 0x28
Filename: /data/dalvik-cache/x86_64/system<at:var at:name="framework"
/>boot-conscrypt.art, /data/dalvik-cache/x86_64/system<at:var at:name
="framework" />boot-conscrypt.art
flags: 0, 0x0
ret: 0x28
Filename: /data/dalvik-cache/x86_64/system<at:var at:name="framework"
/>boot-okhttp.art, /data/dalvik-cache/x86_64/system<at:var at:name="fra
mework" />boot-okhttp.art
flags: 0, 0x0
ret: 0x28
...

```

Interceptor & Backtrace

- Interceptor로 임의 함수 후킹 후, `Thread.backtrace` 사용하여 함수 호출 과정 파악 가능
- 앱 분석 과정에서 실행 흐름을 동작으로 파악 목적

```
var open = Module.findExportByName(null, "open");
// 현재 프로세스에서 open 함수를 찾아서 Open 변수에 저장

Interceptor.attach(open, {
// open 함수 호출 시 해당 호출 가로채고, 실행할 콜백 정의
  onEnter(args) {
    console.log('called from:\n' +
      Thread.backtrace(this.context, Backtracer.FUZZY).map(DebugSymbol.fromAddress).join('\n') + '\n');
// onEnter는 open 함수가 호출될 때 실행되는 콜백
// args는 open함수의 인자가 담긴 배열. open 함수 호출 시 발생할 일 정의
// 현재 스레드에서 호출 스택(백트레이스)를 가져옴. this.context 함수에 백트레이스 생성
// Backtracer.FUZZY는 backtrace를 더 유연하게 추적
// map(DebugSymbol.fromAddress)는 스택 각 주소의
// 디버그 심볼로 변환하여 해당 함수 이름과 소스 코드 위치 추적
// join('\n')는 심볼을 줄바꿈으로 연결하여여러 줄로 출력
  },
  onLeave(retval) {
    console.log("ret: " + retval);
  }
// open 함수 호출 완료된 후 실행되는 콜백
// retval : 함수의 반환 값
});
```

결과

```
$ frida -D emulator-5554 -f com.google.android.calendar -l backtrace.js
```

```

┌───┐
│ _ │ Frida 16.1.3 - A world-class dynamic instrumentation toolkit
│ (─┘

```

```

> _ | Commands:
/_/ |_ | help → Displays the help system
.... object? → Display information about 'object'
.... exit/quit → Exit
....
.... More info at https://frida.re/docs/home/
Spawned `com.google.android.calendar`. Resuming main thread!
[Android Emulator 5554::com.google.android.calendar] → called from:
0x74f91ce8d628 libc.so!fopen+0x48
0x74f91d15b179 libcutils.so!0xc179
0x74f920117949
0x74f91ce7cc8a libc.so!__sfvwrite+0x19a
0x74f91ce72d59 libc.so!__vfprintf+0x3d39
0x74f91ea1736f libandroid_runtime.so!0xed36f
0x74f91ce6df96 libc.so!bsearch+0x36
0x74f8829eac21 frida-agent-64.so!0x264c21
0x74f8828f049e frida-agent-64.so!0x16a49e
0x74f882966d52 frida-agent-64.so!0x1e0d52
0x74f8829ec681 frida-agent-64.so!0x266681
0x74f8829f0c54 frida-agent-64.so!0x26ac54
0x74f8828f0cba frida-agent-64.so!0x16acba
0x74f8829eebfb frida-agent-64.so!0x268bfb
0x74f88299f5b3 frida-agent-64.so!0x2195b3
0x74f8829f0c54 frida-agent-64.so!0x26ac54

ret: 0x28
called from:
0x74f89a87aad7 libart.so!_ZN9unix_file6FdFileC2ERKNSt3__112basic_string
lcNS1_11char_traitslcEENS1_9allocatorlcEEEEijb+0x77
0x74f89a8797e1 libart.so!_ZN3art2OS17OpenFileWithFlagsEPKcib+0xe1
0x74f89a879bde libart.so!_ZN3art10LockedFile4OpenEPKcibPNSt3__112ba
sic_stringlcNS3_11char_traitslcEENS3_9allocatorlcEEEE+0x6e
0x74f91cec061a libc.so!je_tcache_alloc_small_hard+0x1a
0x74f91ceafffa libc.so!je_malloc+0x33a
0x74f91ceafd05 libc.so!je_malloc+0x45
0x74f91ceb4681 libc.so!je_free+0x71
0x74f91d33950a libc++.so!_ZNSt3__112basic_stringlcNS1_11char_traitslcEEN
S_9allocatorlcEEE6appendEPKcm+0x8a

```

```

0x74f89a879b69 libart.so!_ZN3art10LockedFile4OpenEPKcPNSt3__112basic_stringlcNS3_11char_traitslcEENS3_9allocatorlcEEEE+0x19
0x74f89a72082e libart.so!_ZN3art16OatFileAssistant4LockEPNSt3__112basic_stringlcNS1_11char_traitslcEENS1_9allocatorlcEEEE+0xee
0x74f89a729769 libart.so!_ZN3art14OatFileManager19OpenDexFilesFromOatEPKcP8_jobjectP13_jobjectArrayPPKNS_7OatFileEPNSt3__16vectorINSB_12basic_stringlcNSB_11char_traitslcEENS1_9allocatorlcEEEEENS1_11char_traitslcEENS1_9allocatorlcEEEE+0x159
0x74f89a6dfb92 libart.so!_ZN3artL25DexFile_openDexFileNativeEP7_JNIEnvP7_jclassP8_jstringS5_iP8_jobjectP13_jobjectArray+0xb2
0x74f89a85664b libart.so!_ZN3artL37JniMethodEndWithReferenceHandleResultEP8_jobjectjPNS_6ThreadE.llvm.935100503+0x3b
0x72a25e0e boot-core-libart.oat!0x77e0e
0x722fdd5c boot.oat!0x265d5c
0x72ac080b boot-core-libart.oat!0x11280b
...

```

Java

- 대상 프로세스가 JAVA VM을 로드하여 동작 시 사용하며, JAVA 관련 기능 제공
- 안드로이드 앱 후킹 시 사용
- **Java.perform(fn)**
 - 현재 frida 스레드를 JAVA VM에 attach 하고, fn 인자로 전달된 함수 호출
 - fn 함수는 자바 문법이 아닌 자바스크립트 형태 문법으로 작성 되어도 정상적 작동
- **Java.use(className)**
 - 클래스 이름을 인자로 전달 받아 해당 클래스를 자바스크립트에서 사용할 수 있도록 Wrapper 제공
 - Wrapper 통해 클래스 내 메소드 자유롭게 호출 및 후킹
 - Wrapper

- 기존 클래스나 객체를 감싸 더 많은 기능 추가

- **Java.choose(className, callbacks)**

- 자바 힙 메모리 내에서 인자로 전달된 클래스가 이미 객체화되어 있는 객체
- callbacks 인자로 전달된 콜백에서 객체를 찾았을 때와 검색이 완료 되었을때 콜백 추가

- **Java.vm**

- JavaVM 객체 반환
- JNI 함수 후킹 목적이며 JNIEnv 구조체 주소 얻어올 때 사용
- JNIEnv 주소는 `Java.vm.getEnv()` 통해 얻음

```
Java.perform(function () {
// Frida에서 java환경에 접근하여 작업 수행하는 함수
    setTimeout(() => {
// JavaScript에서 주어진 시간(밀리초) 후에 지정된 함수를 실행하는 타이머 함수
        makeToast();
    }, 200)
// setTimeout 끝난 후 호출 함수
// setTimeout의 타이머가 200ms 후에 makeToast() 호출하도록 설정 코드
})

function makeToast(){
// Android에서 Toast 메시지 띄움
    var context = Java.use('android.app.ActivityThread').currentApplication
    ().getApplicationContext();
// Java.use는 Frida에서 특정 Java 클래스 가져오는 메서드
// 'android.app.ActivityThread'는 Android 내부 클래스
// currentApplication() 메서드를 사용하여 현재 실행 중인 애플리케이션의 Context 객체 불러옴
    var toast = Java.use("android.widget.Toast");
// Java.use("android.widget.Toast")는 Toast 클래스를 가져옴
// Toast는 Android에서 사용자에게 간단한 메시지 띄울때 사용 UI 요소
```

```

// Toast 변수는 android.widget.Toast 클래스의 인스턴스를 생성하고 해당 클래스의 메서드를 사용할 수 있게 함
// toast.makeText(context, Java.use("java.lang.String").$new("Welcome to Dreamhack"), 1).show();
Java.scheduleOnMainThread(function() { toast.makeText(context, Java.use("java.lang.String").$new("Welcome to Dreamhack"), 1).show(); })
};
// Java.scheduleOnMainThread(function() :
// Frida에서 UI 관련 작업을 메인 스레드에서 실행할 수 있도록 예약하는 메서드
// Toast.makeText(context, Java.use("java.lang.String").$new("Welcome to Dreamhack"), 1).show() :
// show()를 통해 Toast를 생성하고 화면에 표시
// makeText는 context와 "Welcome to Dreamhack" 문자열을 받아 Toast 메시지 생성
// 1은 짧은 시간 동안 메시지를 표시하는 상수

```

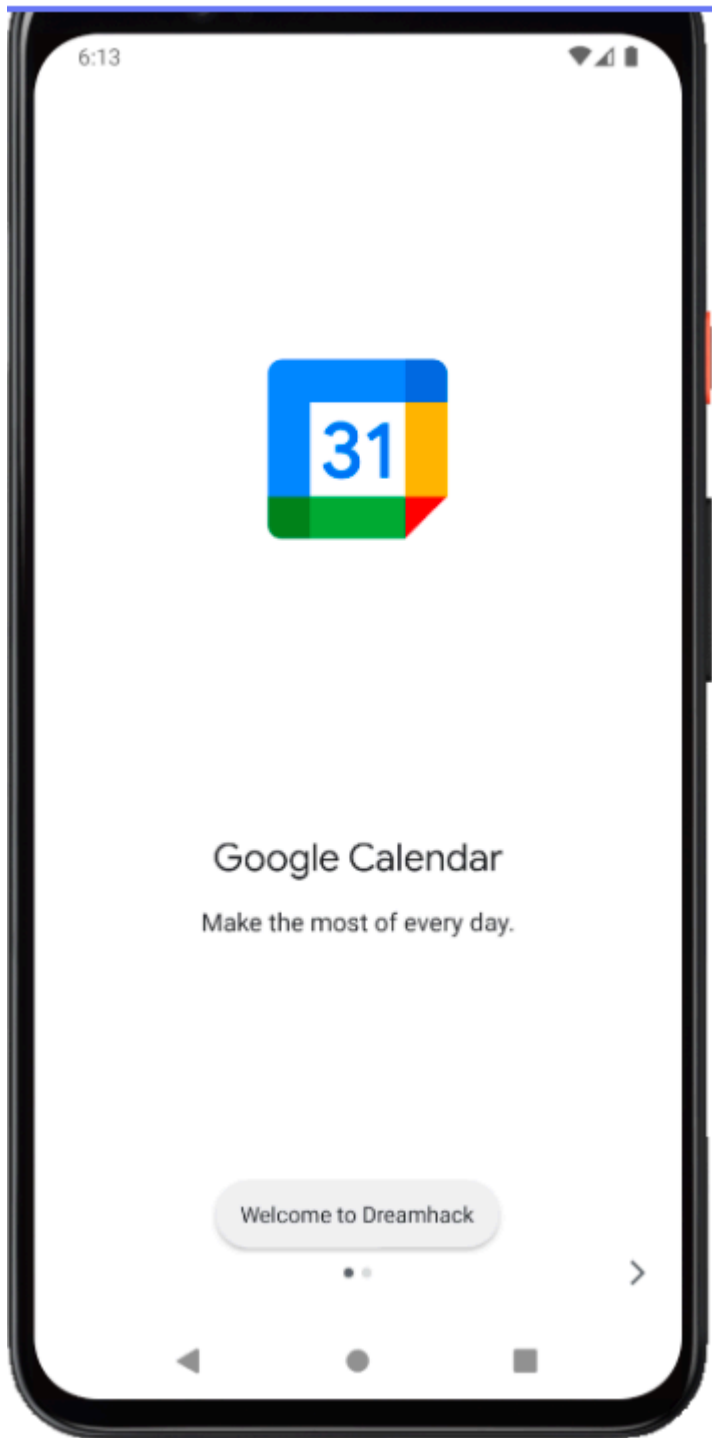
결과

```

$ frida -D emulator-5554 -f com.google.android.calendar -l java.js

_
/_ | Frida 16.1.3 - A world-class dynamic instrumentation toolkit
|(_|
>_ | Commands:
/_/_| help    → Displays the help system
.... object? → Display information about 'object'
.... exit/quit → Exit
....
.... More info at https://frida.re/docs/home/
Spawned `com.google.android.calendar`. Resuming main thread!
[Android Emulator 5554::com.google.android.calendar] →

```



ObjC

- Objective-C 클래스를 로드하는 macOS 또는 iOS 앱에서 사용하여 Objective-C 정보 제공

- **ObjC.available**
 - 대상 프로세스에 Objective-C 클래스 로드 확인 기능
 - macOS & iOS 앱 여부 판단
- **ObjC.api**
 - ObjC 클래스의 메소드 사용할 수 있도록 주소 반환
- **ObjC.classes**
 - ObjC의 클래스 정보 모두 반환
 - 다양한 함수를 앱 내 실행 가능
- **ObjC.schedule[queue, work]**
 - queue에서 지정한 대기열에 work 자바스크립트 함수를 예약

```
var UIAlertView = ObjC.classes['UIAlertView'];
// iOS의 기본 알림 대화상자 클래스 UIAlertView라는 Objective-C 클래스를 가져오는 Frida의 API
ObjC.schedule(ObjC.mainQueue, function () {
// ObjC.schedule : main queue에서 콜백 함수를 실행하도록 예약
// ObjC.mainQueue : 메인 큐를 나타내며, 실행할 코드가 메인 스레드에서 실행되도록 보장
    var view = UIAlertView.alloc().initWithTitle_message_delegate_cancelButtonTitle_otherButtonTitles_(
// UIAlertView.alloc()는 UIAlertView의 인스턴스 생성 코드
// .initWithTitle_message_delegate_cancelButtonTitle_otherButtonTitles는
// UIAlertView 초기화 메서드로, 대화상자의 제목 & 메시지
// & 대리자 & 취소 버튼 & 제목 & 기타 버튼을 인자로 받음
        "Dreamhack",
// UIAlertView의 제목에 해당하는 문자열, 여기서는 Dreamhack이란 제목 설정
        "Welcome from Dreamhack",
// UIAlertView의 메시지 설정
```



```

    NULL,
    // 대리자 (delegate) 나타내는 인자로 미설정
    "OK",
    // 취소 버튼 제목. OK라는 텍스트를 취소 버튼에 설정
    NULL);
    // 기타 버튼의 제목 나타내는 인자. 추가 버튼 없으므로 NULL로 설정
view.show();
// view.show()는 UIAlertView를 화면에 표시하는 코드
// 이 호출로 알림 대화상자가 화면에 표시됨
view.release();
// 메모리 해제를 위한 코드
// UIAlertView는 자동 참조 카운팅을 사용하므로, 사용 후에 release 호출하여 메모리 해제
});

```

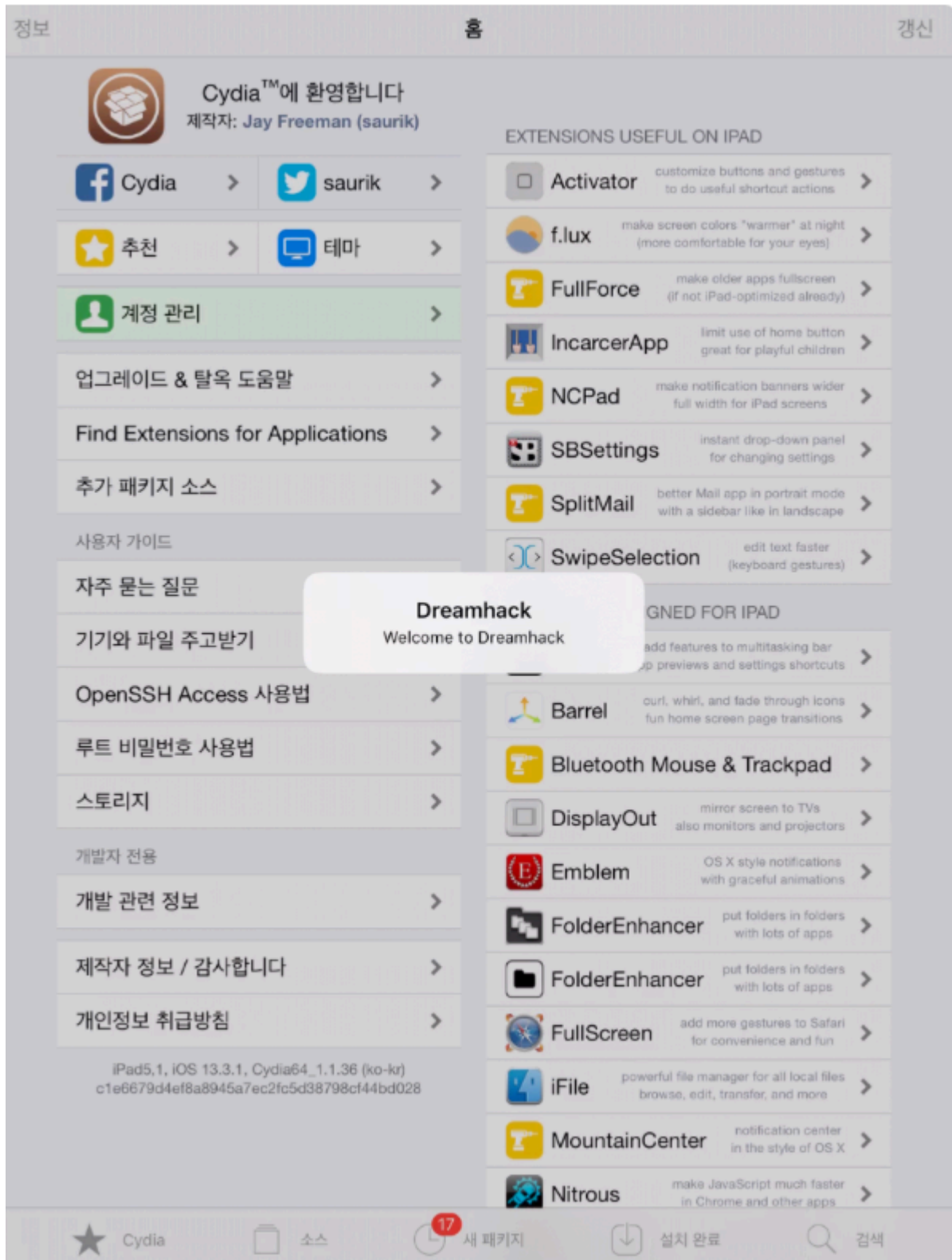
결과

```
$ frida -U -F -l objc.js
```

```

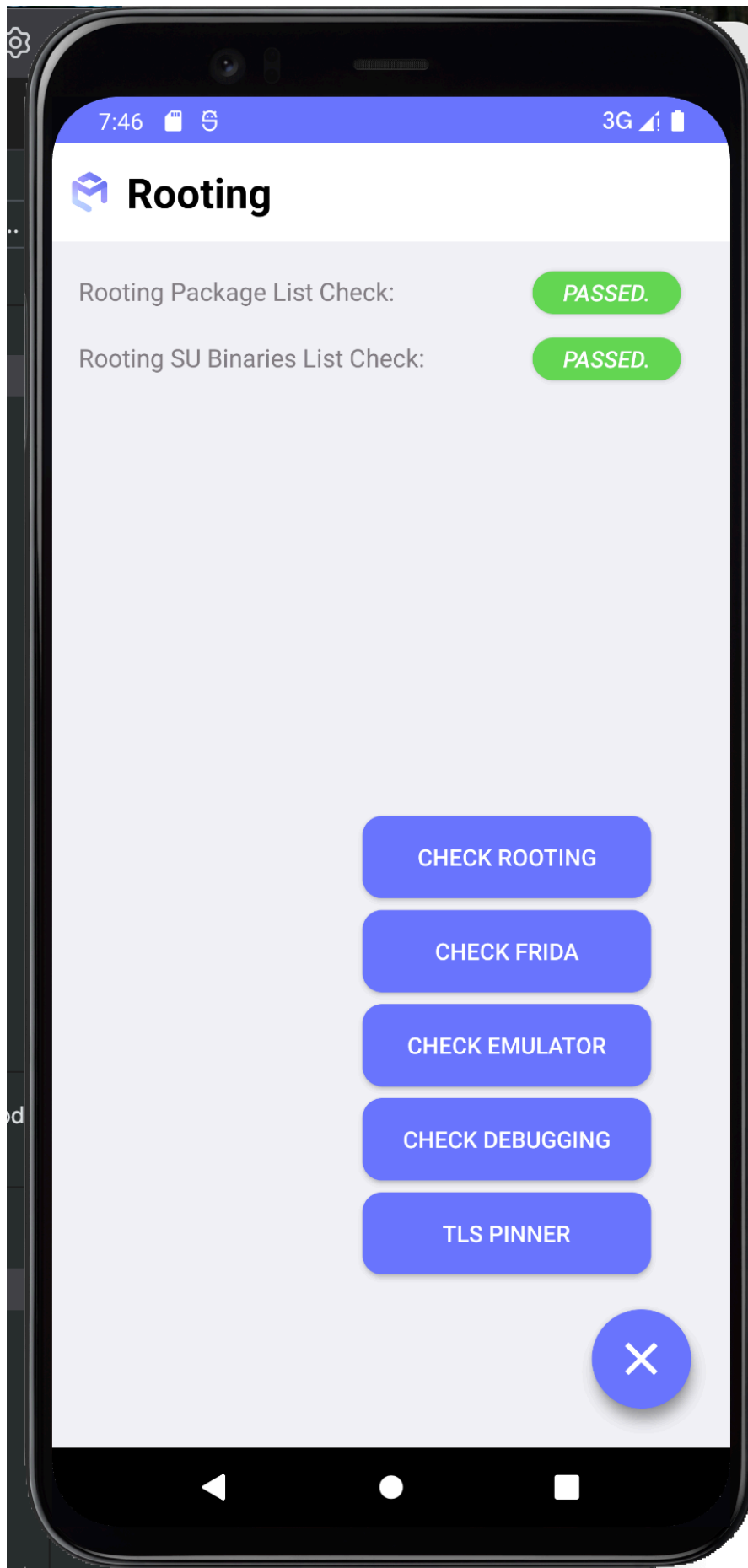
____
/_ |  Frida 16.1.3 - A world-class dynamic instrumentation toolkit
|(_|
>_ |  Commands:
/_|_|  help    → Displays the help system
....  object?  → Display information about 'object'
....  exit/quit → Exit
....
....  More info at https://frida.re/docs/home/
[iPad::Cydia] →

```

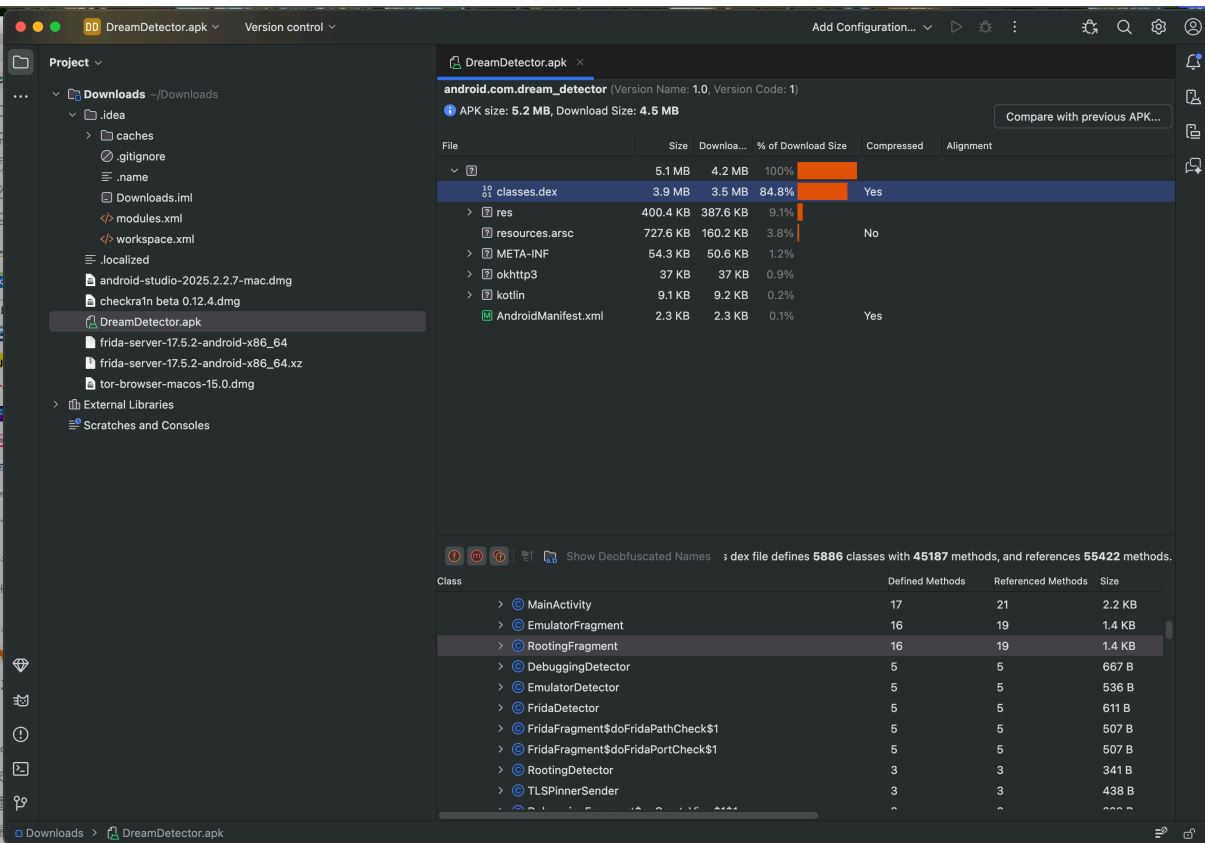




Rooting 패키지 검사

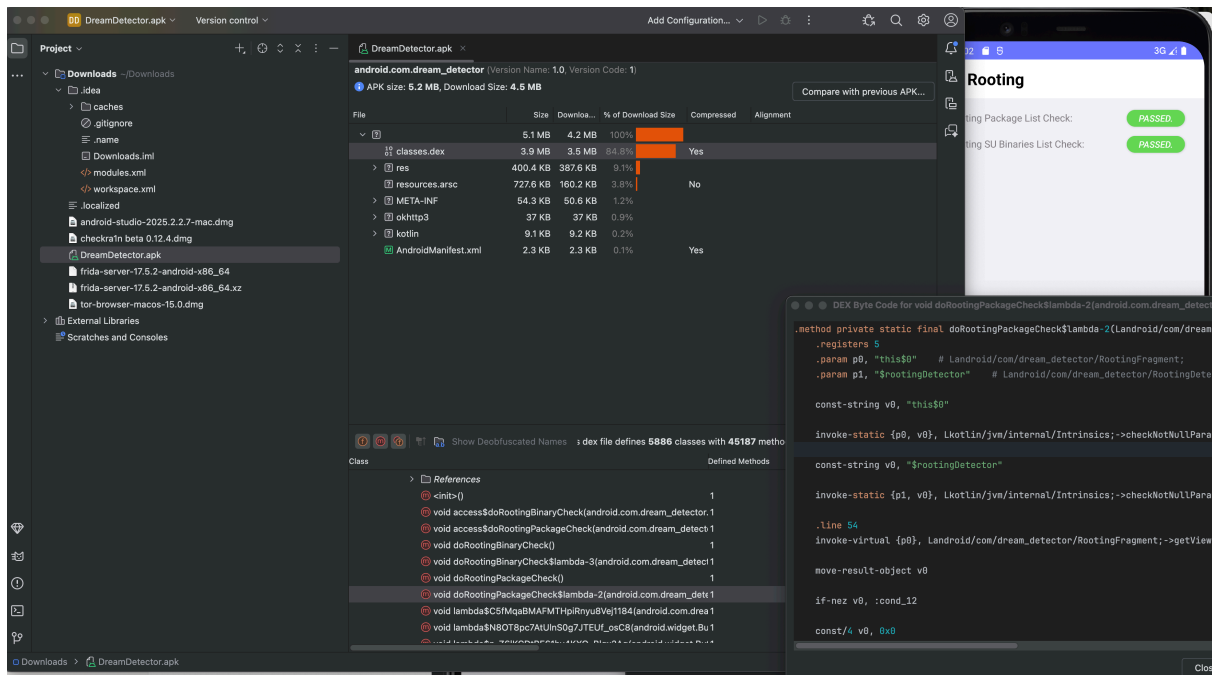


Check Rooting



DreamDetector.apk를 열면 classes.dex → android → com → dream_detector → Rootingfragment

위치에 Check Rooting 구성 요소가 존재



void doRootingPackageCheck\$lambda-2를 누르면

해당 코드 동작이 보이고, kotlin으로 변경하면 아래의 코드와 같음

```
private fun doRootingPackageCheck() {
    // RootingFragment 안에 있는 private함수
    // 외부에서 직접 호출하지 못하고 버튼 클릭 이벤트 같은 내부 로직에서만 호출
    val rootingDetector = RootingDetector(context)
    // context를 value값으로 rootingDetector 변수에 저장
    handler.post {
        // 안드로이드 UI변경은 메인 스레드에서만 가능하므로 halder.post문으로 캡슐
        화
        val t = view?.findViewById<Button>(R.id.btnRootingPackagesChec
        k)

        // RootingPackagesCheck 옆에 있는 버튼을 찾음
        if (rootingDetector.checkRootingPackage()) {
            // 루팅 앱 패키지 설치 시 true
            t?.text = resources.getString(R.string.msg_Detected)
            // 루팅 패키지가 탐지된 경우 버튼 텍스트를 "DETECTED"로 변경
            t?.setBackgroundResource(R.drawable.button_detected)
            // 버튼 배경을 빨간색으로 변경
        } else {
            t?.text = resources.getString(R.string.msg_Passed)
            // 루팅 패키지가 탐색되지 않은 경우 버튼 텍스트를 "PASSED"로 변경
        }
    }
}
```

```

        t?.setBackgroundResource(R.drawable.button_passed)
        // 버튼 배경을 초록색으로 변경
    }
}
}

```

```

public boolean checkRootingPackage() {
    ArrayList<String> packages = new ArrayList<>(Arrays.asList(Constants.
knownRootAppsPackages));
    // 루팅 앱 패키지명 블랙리스트
    PackageManager pm = mContext.getPackageManager();
    // Android 시스템 패키지 관리 객체 획득
    for (String packageName : packages) {
        // 루팅 앱 패키지 리스트 하나씩 순회
        try {
            // 예외 처리 시작. 미설치 앱 조회하면 예외 발생하기 때문
            pm.getPackageInfo(packageName, 0);
            // 핵심 검사 코드
            // 앱 설치 시 정상 실행
            return true;
            // 하나라도 루팅 앱 패키지 찾을 시 도달
            // 루팅 탐지 (DETECTED) 버튼 출력 및 true 반환
        } catch (PackageManager.NameNotFoundException ignore) {
            // 해당 패키지 미설치 시 다음 패키지 이름으로 계속 검사
        }
    }
    return false;
    // 루팅 앱 미설치시 도달. 루팅 흔적 없음 (PASSED) 출력.
}

```

```

public static final String[] knownRootAppsPackages = {
    // 정의된 루팅 패키지 리스트
    "com.noshufou.android.su",
    "com.noshufou.android.su.elite",
    "eu.chainfire.supersu",
    "com.koushikdutta.superuser",
    "com.thirdparty.superuser",
}

```

```
"com.yellowes.su",  
"com.topjohnwu.magisk",  
"com.kingroot.kinguser",  
"com.kingo.root",  
"com.smedialink.oneclickroot",  
"com.zhiqupk.root.global",  
"com.alephzain.framaroot",  
"com.koushikdutta.rommanager",  
"com.koushikdutta.rommanager.license",  
"com.dimonvideo.luckypatcher",  
"com.chelpus.lackypatch",  
"com.ramandroid.appquarantine",  
"com.ramandroid.appquarantinepro",  
"com.android.vending.billing.InAppBillingService.COIN",  
"com.android.vending.billing.InAppBillingService.LUCK",  
"com.chelpus.luckypatcher",  
"com.blackmartalpha",  
"org.blackmart.market",  
"com.allinone.free",  
"com.repodroid.app",  
"org.creeplays.hack",  
"com.baseappfull.fwd",  
"com.zmapp",  
"com.dv.marketmod.installer",  
"org.mobilism.android",  
"com.android.wp.net.log",  
"com.android.camera.update",  
"cc.madkite.freedom",  
"com.solohsu.android.edxp.manager",  
"org.meowcat.edxposed.manager",  
"com.xmodgame",  
"com.cih.game_cih",  
"com.charles.lpoqasert",  
"catch_me_if_you_can_",  
"com.devadvance.rootcloak",  
"com.devadvance.rootcloakplus",  
"de.robv.android.xposed.installer",  
"com.saurik.substrate",
```



```
"com.zachspontemprootremovejb",  
"com.amphoras.hidemyroot",  
"com.amphoras.hidemyrootadfree",  
"com.formyhm.hiderootPremium",  
"com.formyhm.hideroot"  
};
```

DreamDangerous.apk 실행

adb의 경로 찾기

```
ls ~/Library/Android/sdk/platform-tools/adb
```

결과 : /Users/mac/Library/Android/sdk/platform-tools/adb

환경 변수 설정

```
echo 'export ANDROID_HOME=$HOME/Library/Android/sdk' >> ~/.zshrc  
echo 'export PATH=$PATH:$ANDROID_HOME/platform-tools' >> ~/.zshrc  
source ~/.zshrc  
adb version
```

결과 :

Android Debug Bridge version 1.0.41

Version 36.0.0-13206524

Installed as /Users/mac/Library/Android/sdk/platform-tools/adb

Running on Darwin 24.6.0 (x86_64)

adb devices 찾기

```
adb devices
```

결과 :

```
List of devices attached  
emulator-5554 device
```

DreamDangerous.apk 설치

```
adb install -r DreamDangerous.apk
```

결과 :

```
Performing Streamed Install  
Success
```

dreamDangerous의 패키지 경로 찾기

```
adb shell pm list packages -2  
// 근래 설치된 패키지 2개 리스트 뽑기
```

결과 :

```
package:android.com.dream_detector  
package:com.noshufou.android.su
```

해당 패키지 웹 주소 얻기

```
adb shell pm path com.noshufou.android.su
```

결과 :

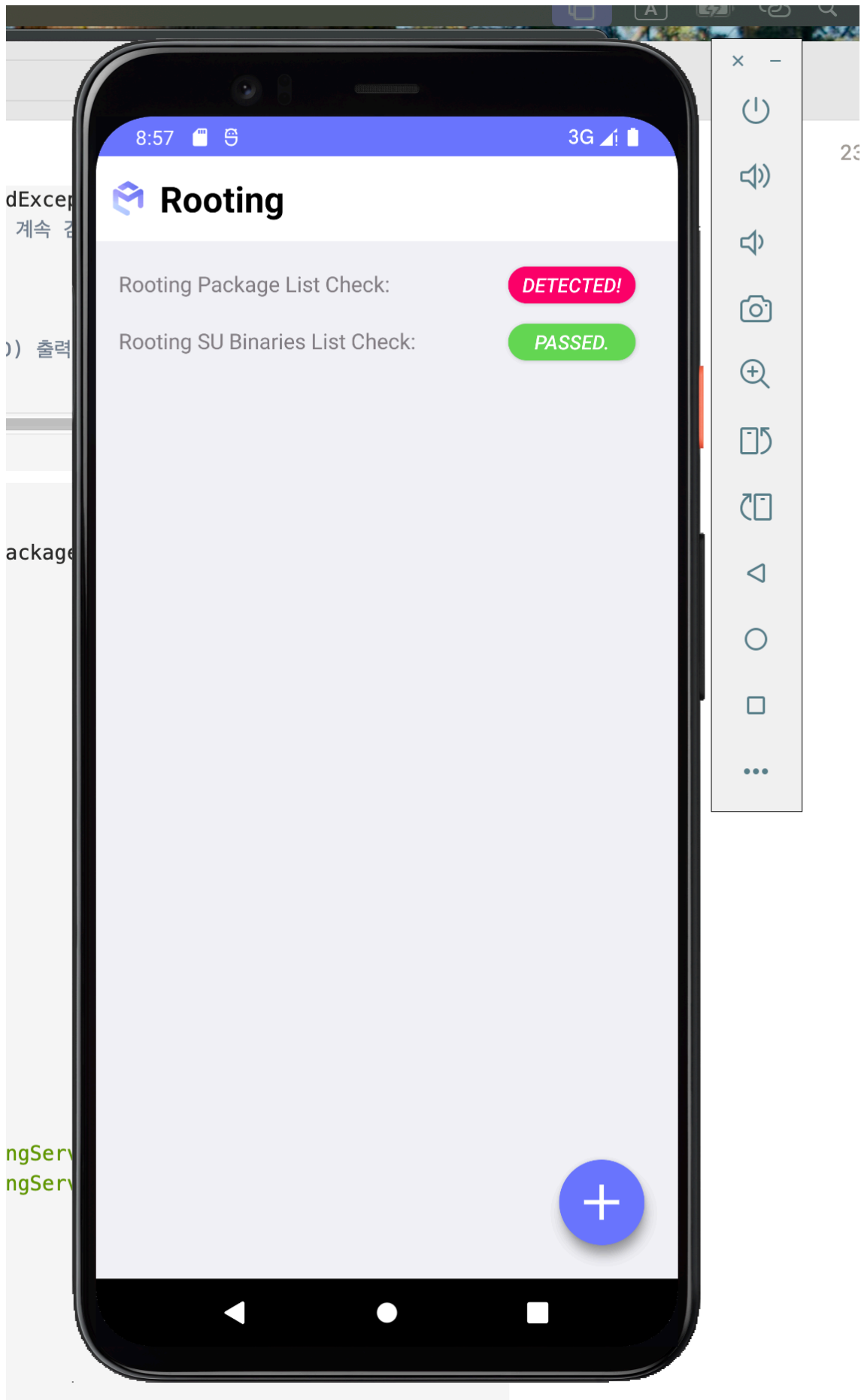
```
package:/data/app/~~dPCpS3Gs3U43Pm2dQBFgkQ==/com.noshufou.and  
roid.su-w4hsr5a0z4V19TEb_gj4fw==/base.apk
```

셸 실행

```
adb shell ls /data/app/~~dPCpS3Gs3U43Pm2dQBFgkQ==/com.noshufou.a  
ndroid.su-w4hr5a0z4V19TEb_gj4fw==/base.apk
```

결과 :

```
/data/app/~~dPCpS3Gs3U43Pm2dQBFgkQ==/com.noshufou.android.su-w  
4hr5a0z4V19TEb_gj4fw==/base.apk
```



Rooting Package List Check에 **DETECTED!** 가 출력

루팅 패키지 탐지 성공

```
public boolean checkRootingPackage() {
    ArrayList<String> packages = new ArrayList<>(Arrays.asList(Constants.
knownRootAppsPackages));
    // 루팅 앱으로 알려진 패키지명 목록 리스트를 packages 변수에 저장

    PackageManager pm = mContext.getPackageManager();
    // Android 시스템 객체에 패키지 관리자 객체를 저장

    for (String packageName : packages) {
        // 루팅 패키지 리스트를 하나씩 순회
        try {
            pm.getPackageInfo(packageName, 0);
            // 핵심 검사 코드
            // 설치 시 아래 줄로 진행
            return true;
            // 도달 시 루팅 앱 하나 이상 탐지 의미
        } catch (PackageManager.NameNotFoundException ignore) {
            // 해당 패키지 미설치 시 ignore(무시)하고 다음 패키지를 검사
        }
    }
    return false;
    // 루팅 패키지 미탐지 시 도달
}
```

우회 스크립트 작성

```
function modifyCheckPackageRet() {
    // 루팅 패키지 검사 함수의 "리턴값" 조작 목적 함수 정의
    Java.perform(function() {
        // Frida에서 Java VM 접근 목적 함수
```

```

var RootingDetector = Java.use("android.com.dream_detector.Rooting
Detector");
// 앱 내부의 RootingDetector 클래스를 Frida에 로드
// 내부에 public boolean checkRootingPackages() 함수가 존재
RootingDetector.checkRootingPackage.implementation = function() {
// checkRootingPackage() 매서드를 통째로 가로채기
// Java를 Javascript 함수로 덮어 씌움
return false;
// 무조건 false 반환. 루팅 앱 설치 여부를 떠나 passed 처리
};
});
}

function modifyKnownPackagesVal() {
// 루팅 패키지의 "블랙리스트 자체"를 조작하는 함수
// getPackageInfo의 인자로 인자를 전달하면 의미 없는 배열을 추가하여 PASSED
출력토록 함
Java.perform(function() {
var Constants = Java.use("android.com.dream_detector.Constants");
// 앱 내부의 Constants 클래스를 Frida에 로드
// public static String[] knownRootAppsPackages가 내부에 존재
var str_arr = Java.array('java.lang.String',[""]);
// Java의 String[] 타입 배열 새로 생성
Constants.knownRootAppsPackages.value = str_arr;
// 기존 루팅 앱 패키지 블랙리스트를 위 배열에 로드
});
}

modifyCheckPackageRet();
// 리턴값 조작 우회 함수 실행
modifyKnownPackagesVal();
// 블랙리스트 조작 우회 함수 실행

```

해당 내용을 `Bypass_Rooting_checkPackage.js` 라는 파일로 저장

```
nano ~/Downloads/Bypass_Rooting_checkPackage.js
```

서버 연결

```
adb push frida-server-17.5.2-android-x86_64 /data/local/tmp/frida-server
adb shell chmod 755 /data/local/tmp/frida-server
adb shell /data/local/tmp/frida-server
// 포그라운드 상태이므로 터미널을 하나 더 띄워 아래 실행 명령어 실행
```

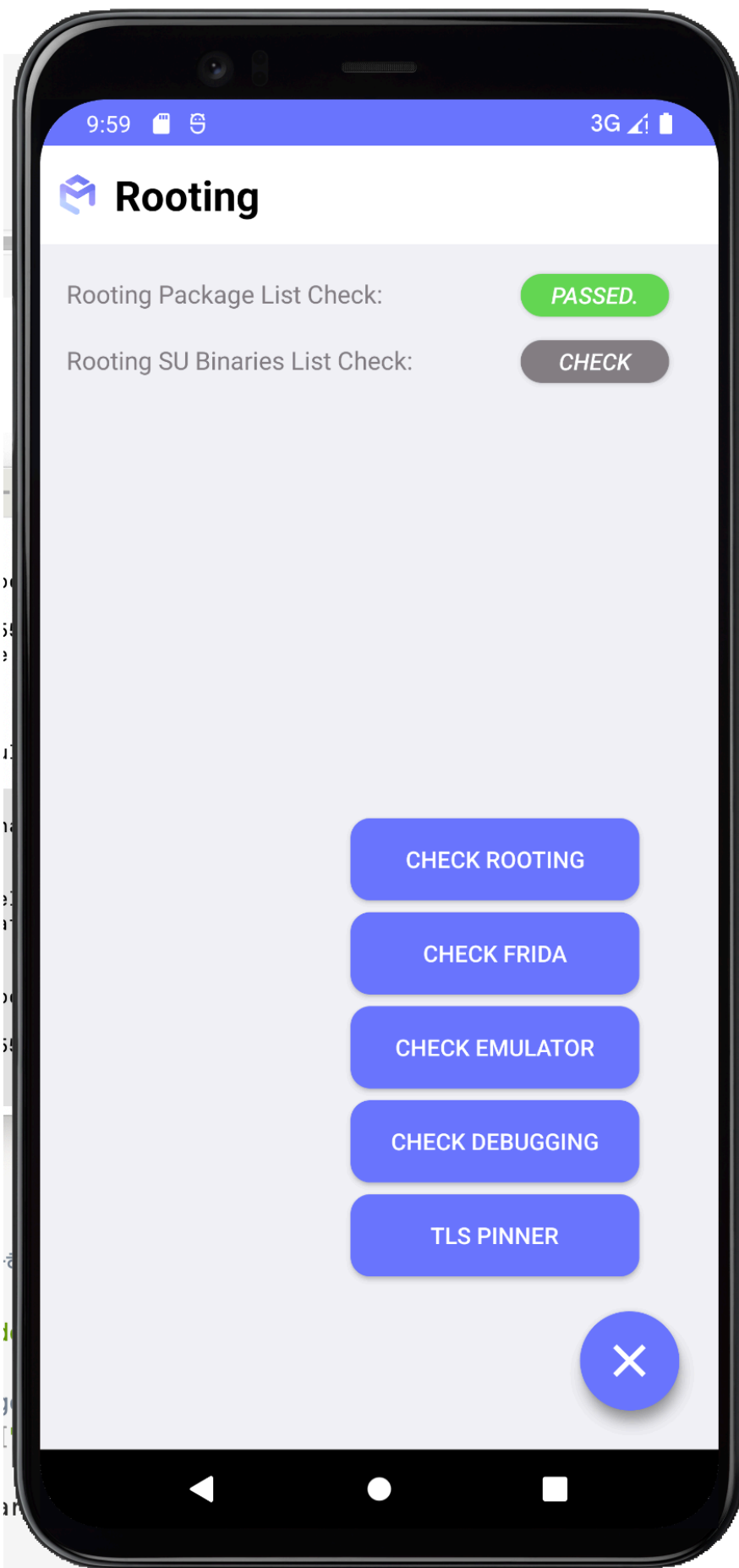
실행 명령어

```
frida -D emulator-5554 -F -l ./Bypass_Rooting_checkPackage.js
```

결과

```

┌───┐
/_ |  Frida 17.5.2 - A world-class dynamic instrumentation toolkit
|(_| |
>_ |  Commands:
/_/ |_|    help    → Displays the help system
....    object?  → Display information about 'object'
....    exit/quit → Exit
....
....    More info at https://frida.re/docs/home/
....
....    Connected to Android Emulator 5554 (id=emulator-5554)
[Android Emulator 5554::Dream-detector ]→
```





Rooting 파일 검사 우회

checkSuBinary

- 버튼 클릭에 의해 동작하는 UI 관련 함수

```
class RootingFragment : Fragment() {  
    // 루팅 검사&체크 화면 담당하는 클래스  
    val handler = Handler(Looper.getMainLooper())  
    // 메인 UI 스레드 용도 handler를 handler 변수에 로드  
    ...  
    @SuppressWarnings("ResourceAsColor")  
    // 정적 분석(Lint) 경고 억제용 어노테이션  
    // 리소스 색상 사용 방식 경고 무시  
    private fun doRootingBinaryCheck() {  
        // su 바이너리 존재 여부 검사 함수  
        // private > Fragment 내부에서만 사용  
        // 버튼 클릭시 호출  
        val rootingDetector = RootingDetector(context)  
        // 파일 시스템 접근 & 시스템 정보 조회 & 루팅 흔적 검사를 위해 context를  
        // rootingDetector 변수에 로드  
        handler.post {  
            // UI 스레드에서 실행될 작업 예약  
            val t = view?.findViewById<Button>(R.id.btnRootingBinariesCheck)  
            // 현재 Fragment의 view에서 "Rooting SU Binaries List Check" 버튼 찾  
음  
            if (rootingDetector.checkSuBinary()) {  
                // checkSuBinary는 바이너리 파일 존재 여부 검사 함수로 아래 리스트가 해  
당 대상  
                // /system/bin/su, /system/xbin/su, /sbin/su  
                // true일 시 루팅 탐지, false일 시 su 바이너리 미존재  
                t?.text = resources.getString(R.string.msg_Detected)  
                // su 바이너리 발견 시 버튼에 "DETECTED!" 변경
```

```

        // 실제 문자열은 resvalues/strings.xml에 정의
        t?.setBackgroundResource(R.drawable.button_detected)
        // su 바이너리 발견 시 버튼 색상 빨간색으로 변경
    } else {
        t?.text = resources.getString(R.string.msg_Passed)
        // su 바이너리 미발견 시 버튼에 "PASSED" 출력
        t?.setBackgroundResource(R.drawable.button_passed)
        // su 바이너리 미발견 시 버튼 색상 초록색 출력
    }
}
}
}
}

```

checkSuBinary 함수

- 경로
 - `app/src/main/java/android/com/dream_detector/RootDetector.java`

```

public boolean checkSuBinary() {
    for (String directory : Constants.knownSuDirectories) {
        // 정의된 경로로 반복문이 돌며 하나씩 directory에 저장
        for (String filename : Constants.knownSuBinaries) {
            // 저장된 탐지할 파일 이름을 반복문을 돌며 하나씩 filename 변수로 불러와, directory와
            // 결합하여 온전한 파일 경로 f 라는 파일 객체 생성
            File f = new File(directory, filename);
            if (f.exists()) {
                // 파일 존재 여부 확인 후 파일 존재 시 true qksghks
                return true;
            }
        }
    }
    return false;
}

```

Constants.knownSuBinaries (검사할 파일명 및 파일 경로)

- 경로

- `app/src/main/java/android/com/dream_detector/Constants.java`

```
public static final String[] binaries = {
    "su",
    // user id 통해 권한 바꾸는 실행 파일
    "busybox",
    // unix 유틸리티 제공하는 실행 파일로, root 권한 획득 가능
    "magisk"
    // 앱에 root 권한 접근 & 부트 이미지 리패키징. read-only 저장소 수정
};
```

바이너리 탐지경로 저장 내역

```
public static final String[] binaryPaths = {
    "/data/local/",
    "/data/local/bin/",
    "/data/local/sbin/",
    "/sbin/",
    "/su/bin/",
    "/system/bin/",
    "/system/bin/.ext/",
    "/system/bin/failsafe/",
    "/system/sd/sbin/",
    "/system/usr/we-need-root/",
    "/system/xbin/",
    "/cache/",
    "/data/",
    "/dev/"
};
```

탐지 우회

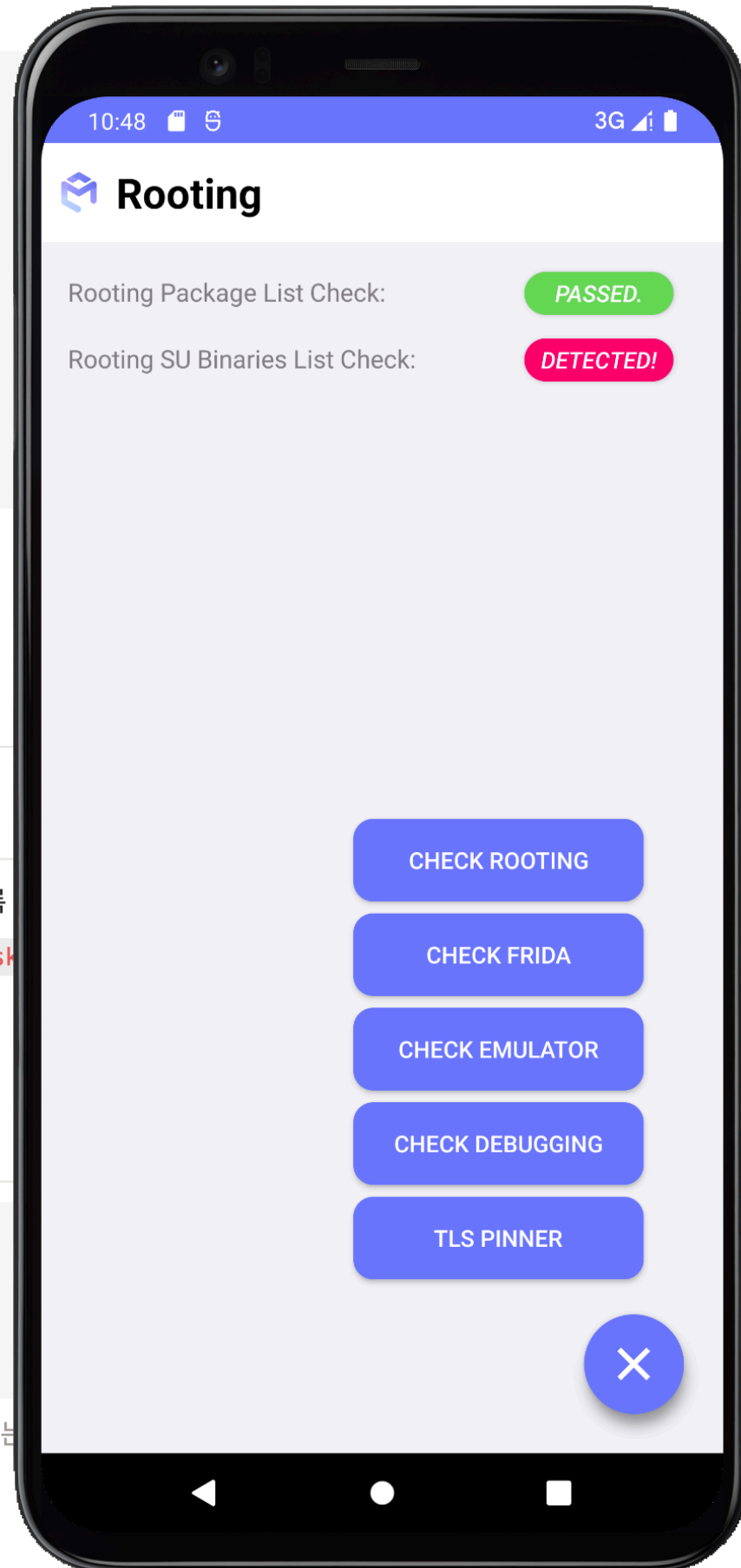
환경 설정

루팅 탐지 기능 우회 전 실습 목적으로 루팅 관련 파일 탐지한 상태 되도록 설정 필요

`Constants.knownSuBinaries` 에 정의된 `busybox` , `su` , `magisk` 파일명을 가진 파일을 생성하여
루팅 파일 검사 기능이 `DETECTED!` 상태 되도록 설정 필요

설정 방법

```
adb shell
touch /data/busybox
```



Rooting SU Binaries List Check 가 DETECTED! 로 변경된 것을 확인 가능

함수 반환값 조작

변경 전

```
public boolean checkSuBinary() {
    for (String directory : Constants.knownSuDirectories) {
        for (String filename : Constants.knownSuBinaries) {
            File f = new File(directory, filename);
            if (f.exists()) {
                return true;
                // true 값을 false로 만들어야 해당 검사 우회 가능
            }
        }
    }
    return false;
}
```

변경 후 (우회 스크립트 작성)

```
function modifyCheckSuBinaryRet() {
    // su 바이너리 검사 함수의 반환값 조작 목적 사용자 정의 함수
    Java.perform(function() {
        // Frida에서 Android 앱의 Java VM에 안전하게 접근 목적 필수 Wrapper
        var RootingDetector = Java.use("android.com.dream_detector.RootingDetector");
        // 앱 내부의 RootingDetector 클래스를 Frida로 로드
        // public boolean checkSuBinary() 함수가 내부에 존재
        RootingDetector.checkSuBinary.implementation = function() {
            // checkSuBinary() 메서드의 원래구현을 덮어쓰기(override)
            // 원래 Java 코드 (File.exists 검사 루프) 전혀 미실행
            return false;
            // 무조건 false 반환
            // /system/bin/busybox가 존재하더라도 없다고 판단하여 UI에 항상 PASS
            ED 표시
        };
    });
}
```

```

    };
  })
}

modifyCheckSuBinaryRet();
// 방금 정의한 우회 함수를 실행

```

실행 코드

```

cd ~/Library/Python/3.9/bin/frida
// frida 위치가 Downloads가 아닌 경우

frida -D emulator -5554 -f android.com.dream_detector -l Bypass_Rooting_
checkfile.js

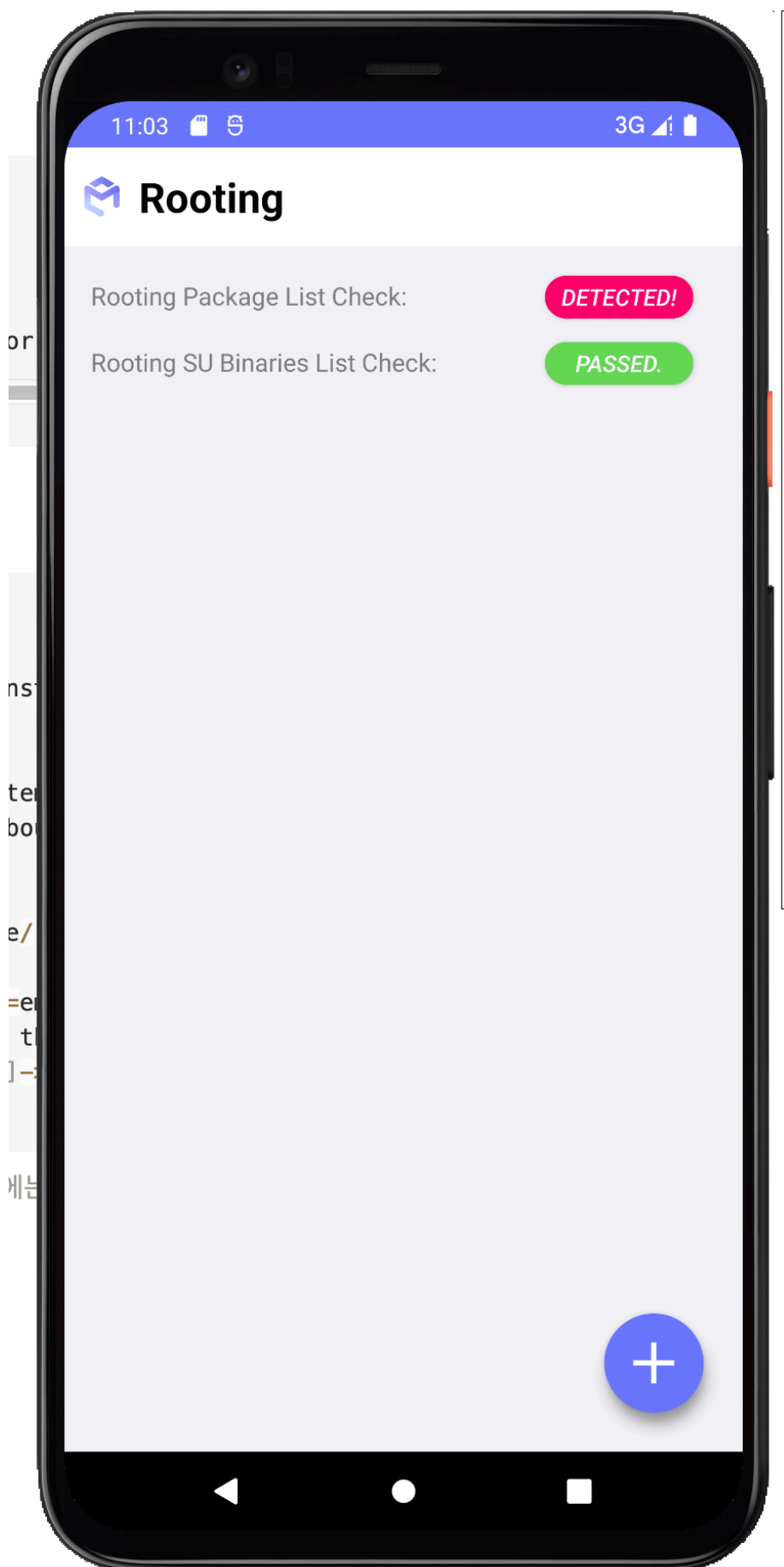
```

결과

```

____
/_ |  Frida 17.5.2 - A world-class dynamic instrumentation toolkit
|(_| |
>_ |  Commands:
/_/ |_|    help    → Displays the help system
....    object?  → Display information about 'object'
....    exit/quit → Exit
....
....    More info at https://frida.re/docs/home/
....
....    Connected to Android Emulator 5554 (id=emulator-5554)
Spawned `android.com.dream_detector`. Resuming main thread!
[Android Emulator 5554::android.com.dream_detector ]→

```

Rooting binaries list check 가 PASSED 상태로 변경된 것을 확인 가능